

Documentation de l'API de recommandation (api.py)

1 API de recommandation (api.py)

Cette section décrit le fonctionnement de l'API FastAPI qui exploite le modèle LightFM entraîné afin de fournir des recommandations de POI au frontend.

2 Fonction recommander_par_liste()

Cette fonction constitue le cœur du moteur de recommandation.

Lorsque l'utilisateur clique sur « **Me recommander** », le frontend envoie une requête HTTP :

```
POST /api/recommend_from_selection

{
  "selected_pois": [123,456,789],
  "user_id": 1
}
```

Étape 1 : Récupération des correspondances

Le fichier `dataset.pkl` contient les correspondances entre les identifiants réels des POI et les index internes utilisés par LightFM.

```
item_mapping = dataset.mapping()[2]
```

Exemple :

<i>POI</i> réel	<i>Index</i> LightFM
123	0
456	1
789	2

ou encore :

<i>POI</i>	<i>Index</i>
<i>TourEiffel</i>	0
<i>MuséeduLouvre</i>	1
<i>ArcdeTriomphe</i>	2

Étape 2 : Conversion des POI en index

Les identifiants sélectionnés par l'utilisateur sont convertis en index internes.

```

item_indices = [
    item_mapping[p]
    for p in selected_pois
]

```

Exemple :

$$[123, 456, 789] \Rightarrow [0, 1, 2]$$

Étape 3 : Récupération des embeddings

Les embeddings appris lors de l'entraînement sont récupérés depuis le modèle.

```

item_embeddings = model.item_embeddings[item_indices]

```

Par exemple :

$$POI_{123} = [0.2, -0.4, 1.1, \dots]$$

$$POI_{456} = [0.5, -0.8, 0.7, \dots]$$

$$POI_{789} = [0.1, -0.3, 1.4, \dots]$$

Ces vecteurs ont été appris lors de l'appel à :

```

model.fit(...)

```

dans `train_lightfm.py`.

Étape 4 : Construction d'un profil utilisateur temporaire

Un profil représentant les préférences actuelles de l'utilisateur est obtenu en calculant la moyenne des embeddings des POI sélectionnés.

```

user_embedding = np.mean(
    item_embeddings,
    axis=0
)

```

Exemple :

$$POI_{123} = [1, 2]$$

$$POI_{456} = [3, 4]$$

$$POI_{789} = [5, 6]$$

Le profil obtenu est :

$$\frac{[1, 2] + [3, 4] + [5, 6]}{3} = [3, 4]$$

Ce vecteur représente les préférences estimées de l'utilisateur.

Étape 5 : Calcul des scores de similarité

Chaque POI connu par le modèle est comparé au profil utilisateur.

```
scores = np.dot(
    model.item_embeddings,
    user_embedding
)
```

Mathématiquement :

$$score = embedding_{POI} \cdot profil_{utilisateur}$$

Exemple :

POI	Score
A	0.91
B	0.88
C	0.23
D	0.95

Plus le score est élevé, plus le POI est considéré comme similaire aux POI sélectionnés.

Étape 6 : Classement des résultats

Les POI sont triés selon leur score décroissant.

```
ranked = np.argsort(-scores)
```

Le classement devient par exemple :

$$D \rightarrow A \rightarrow B \rightarrow C$$

Étape 7 : Suppression des POI déjà sélectionnés

Les POI déjà présents dans la sélection de l'utilisateur sont retirés.

```
ranked = [
    i for i in ranked
    if i not in item_indices
]
```

Cette étape évite de recommander des POI déjà choisis.

Étape 8 : Sélection du Top 10

Seuls les dix premiers résultats sont conservés.

```
top_indices = ranked[:10]
```

Exemple :

45, 82, 13, 67, ...

Étape 9 : Retour aux identifiants réels

Les index internes sont reconvertis en identifiants de POI.

```
recommended_ids = [  
    reverse_mapping[i]  
    for i in top_indices  
]
```

Exemple :

45 → 22055

82 → 28218

13 → 61962

Les identifiants retournés correspondent aux POI présents dans la base de données.

3 Fonction `enrich_pois()`

À ce stade, le moteur possède uniquement les identifiants des POI recommandés.

[22055, 28218, 61962]

Ces identifiants ne sont pas suffisants pour l’affichage dans le frontend.

La fonction :

```
enrich_pois(recommended_ids)
```

interroge MySQL afin de récupérer les informations descriptives.

La requête SQL récupère notamment :

- le nom du POI ;
- la catégorie ;
- la note moyenne ;
- les autres informations utiles à l’affichage.

Exemple de résultat :

```
[  
  {  
    "idPoi":22055,  
    "name":"Le Live",  
    "categorie":"Concert Hall",  
    "note_moyenne":2.6  
  }  
]
```

Ces données sont ensuite renvoyées au frontend.

4 Utilisation de l’API par le frontend

Le frontend exploite plusieurs endpoints de l’API.

Chargement des catégories

```
GET /api/cats
```

Retour :

```
[
  {
    "idCat":1,
    "name":"Restaurant"
  }
]
```

Ces informations permettent de remplir les filtres de catégories.

Chargement des POI d'une catégorie

```
GET /api/poisbycats/1
```

Retour :

```
[
  {
    "idPoi":123,
    "namePoi":"Chez Mario"
  }
]
```

Les POI sont ensuite affichés sur la carte.

Obtention des recommandations

Lorsque l'utilisateur clique sur « **Me recommander** » :

```
POST /api/recommend_from_selection
```

avec

```
{
  "selected_pois":[123,456,789],
  "user_id":1
}
```

Le moteur renvoie une liste enrichie de recommandations :

```
[
  {
    "idPoi":22055,
    "name":"Le Live",
    "categorie":"Concert Hall"
  }
]
```

Optimisation d'itinéraire

```
POST /api/optimize-itinerary
```

Le frontend transmet les dates ainsi que les POI sélectionnés.

Le service renvoie un itinéraire optimisé.

Cette fonctionnalité est indépendante du moteur LightFM.

Consultation des avis

```
GET /api/avisbypoi/123
```

Retour :

```
[
  {
    "idTip":1,
    "content":"Tr s bon restaurant",
    "note":5
  }
]
```

Ces informations sont affichées dans une fenêtre contextuelle du frontend.

5 Résumé

Résumé du fonctionnement de l'API

Lorsqu'un utilisateur sélectionne plusieurs POI sur la carte, l'API convertit ces identifiants en index internes grâce à `dataset.pkl`. Ces index permettent d'accéder aux embeddings appris par LightFM et stockés dans `lightfm_model_hist.pkl`. Un profil utilisateur temporaire est construit en calculant la moyenne des vecteurs des POI sélectionnés. Ce profil est comparé à l'ensemble des POI via un produit scalaire afin d'obtenir un score de similarité. Les meilleurs résultats sont ensuite filtrés, reconvertis en identifiants réels, enrichis à partir de MySQL, puis renvoyés au frontend pour affichage.